Video Script: Creating Custom Seccomp Profiles

Module 3, Video 2 of 3
Estimated Duration: 14-16 minutes
Technical Setup: 1920x1080 @ 30fps, 150% zoom on all screens

Pre-Production Checklist

- [ ] PowerPoint slides (4 slides) ready
- [ ] Terminal with strace installed
- [ ] VS Code with JSON schema for Seccomp
- [ ] nginx container image pulled
- [ ] Python Flask test app prepared
- [ ] Empty JSON template files ready
- [ ] auditd configured (optional, for advanced demo)

VIDEO SCRIPT

SEGMENT 1: Introduction and Motivation (0:00 - 1:30)

[SCREEN: PPT Slide 1 - Title: “Building Custom Seccomp Profiles”]

NARRATION:
“Welcome back! In the previous video, we explored Docker’s default Seccomp profile and saw how it blocks dangerous syscalls. But here’s the thing: the default profile is a compromise. It needs to work for thousands of different applications, so it allows hundreds of syscalls that YOUR specific application might never use.

[TRANSITION: PPT Slide 2 - Diagram showing three security levels]

[SLIDE SHOWS: No Seccomp (300+ syscalls) → Default Profile (~250 allowed) → Custom Profile (~40-60 allowed)]

Every allowed syscall is a potential attack vector. By creating custom Seccomp profiles tailored to your application, you can reduce the attack surface by 80% or more.

In this video, we’ll build custom Seccomp profiles from scratch. You’ll learn to:
- Identify which syscalls your application actually needs using strace
- Write Seccomp profile JSON
- Test and validate your profiles
- Handle multi-process applications

Let’s dive in.”

SEGMENT 2: Understanding the Workflow (1:30 - 3:00)

[SCREEN: PPT Slide 3 - Workflow diagram with 4 steps]

NARRATION:
“Creating custom Seccomp profiles follows a four-step process:

[HIGHLIGHT each step as mentioned:]

Step 1: Discover - Run your application with strace to capture all syscalls it makes
Step 2: Analyze - Extract unique syscalls and remove duplicates
Step 3: Build - Create the Seccomp JSON with those syscalls
Step 4: Test - Run your app with the profile and fix any issues

This is iterative. Your first profile might be too restrictive and break functionality. That’s normal—we’ll debug and refine it.

Let’s build a profile for nginx, a popular web server.”

SEGMENT 3: Step 1 - Discovering Syscalls with strace (3:00 - 6:30)

[SCREEN: Terminal - fullscreen]

NARRATION:
“First, we’ll run nginx under strace to see what syscalls it uses. strace is a diagnostic tool that intercepts and logs all system calls a process makes.”

[TYPE COMMAND:]

```
docker run --rm --name nginx-trace nginx:alpine /bin/sh -c
"strace -c nginx -g 'daemon off;' 2>&1" | head -40
```

NARRATION (while typing):
“Let me break this down:
- We’re running nginx in a container
- `strace -c` gives us a summary count of syscalls
- `nginx -g 'daemon off;'` runs nginx in foreground
- We redirect stderr because strace outputs there”

[PRESS ENTER - output shows syscall summary]

NARRATION:
“This gives us a summary, but we need the actual syscall names. Let’s capture them properly.”

[TYPE COMMAND:]

```
docker run --rm --security-opt seccomp=unconfined nginx:alpine sh -c "apk add strace
&& strace -f -o /tmp/nginx.strace nginx -g 'daemon off;'" &
```

[COMMAND EXPLANATION:]
“- `--security-opt seccomp=unconfined` - disable Seccomp so strace can see everything
- `-f` - follow child processes (nginx spawns workers)

- `-o /tmp/nginx.strace` - output to file
- We run this in background with `&`"

[PRESS ENTER, wait 3 seconds]

NARRATION:
"Now let's send some test traffic to nginx to capture syscalls during actual operation."

[TYPE:]

```
docker exec $(docker ps -q) sh -c "wget -q -O- http://localhost"
```

[WAIT 2 seconds]

[TYPE:]

```
docker stop $(docker ps -q)
```

NARRATION:
"Great, nginx handled a request. Now let's extract the strace data. We'll copy it from the container before it's deleted."

[TYPE - demonstrating the extraction in a new container:]

```
docker run --rm --security-opt seccomp=unconfined nginx:alpine sh -c "
apk add -q strace &&
strace -f -o /tmp/nginx.strace nginx -g 'daemon off;' 2>/dev/null &
sleep 3 && pkill nginx &&
cat /tmp/nginx.strace" > nginx_strace_output.txt
```

[PRESS ENTER, wait for completion]

NARRATION:
"Now we have the complete strace output. Let's extract just the unique syscall names."

[TYPE:]

```
grep -oP '^[^\(]+' nginx_strace_output.txt | sort -u > nginx_syscalls.txt
```

[COMMAND EXPLANATION:]
"This grep command extracts syscall names before the opening parenthesis, then we sort and get unique values."

[TYPE:]

```
cat nginx_syscalls.txt
```

[OUTPUT shows list like: accept4, bind, brk, close, epoll_create, etc.]

NARRATION:
"Perfect! We've identified all the syscalls nginx needs. I count about 45-50 unique syscalls here. Compare that to the 250+ that Docker's default profile allows."

SEGMENT 4: Step 2 - Building the Profile (6:30 - 10:00)

[SCREEN: Code Editor - VS Code split with nginx_syscalls.txt on left, new seccomp-nginx.json on right]

NARRATION:
“Now we’ll create our Seccomp profile JSON. Let me start with the basic structure.”

[BEGIN TYPING in right pane - speak each section as you type:]

```
{
  "defaultAction": "SCMP_ACT_ERRNO",
  "architectures": [
    "SCMP_ARCH_X86_64",
    "SCMP_ARCH_X86",
    "SCMP_ARCH_X32"
  ],
  "syscalls": [
    {
      "names": [],
      "action": "SCMP_ACT_ALLOW"
    }
  ]
}
```

NARRATION (line by line as typing):
“- **defaultAction**: SCMP_ACT_ERRNO - block everything by default with an error
- **architectures**: We support x86_64, x86, and x32
- **syscalls array**: This will hold our allowed syscalls
- **action**: SCMP_ACT_ALLOW - permit these specific syscalls”

[PAUSE at the empty “names” array]

NARRATION:
“Now I need to populate this names array with our syscalls from the strace output. Let me write a quick command to format this.”

[SCREEN: Terminal - bottom 1/3 of screen, code editor still visible above]

[TYPE:]

```
cat nginx_syscalls.txt | awk '{print "      \""$1"\","}' | head -20
```

[OUTPUT shows formatted syscall names with quotes and commas]

NARRATION:
“This formats our syscalls as JSON array elements. Let me get the complete list and copy it.”

[TYPE:]

```
cat nginx_syscalls.txt | awk '{print "      \""$1"\","}' | pbcopy
```

NARRATION:
“Actually, let me do this properly with all syscalls. I’ll create a Python script to build the full profile.”

[SCREEN: Code Editor - new file build_profile.py]

[TYPE PYTHON SCRIPT - explain as you go:]

```
#!/usr/bin/env python3
import json

# Read syscalls from strace output
with open('nginx_syscalls.txt', 'r') as f:
    syscalls = [line.strip() for line in f if line.strip()]

# Build Seccomp profile
profile = {
    "defaultAction": "SCMP_ACT_ERRNO",
    "architectures": [
        "SCMP_ARCH_X86_64",
        "SCMP_ARCH_X86",
        "SCMP_ARCH_X32"
    ],
    "syscalls": [
        {
            "names": syscalls,
            "action": "SCMP_ACT_ALLOW"
        }
    ]
}

# Write to JSON file
with open('seccomp-nginx.json', 'w') as f:
    json.dump(profile, f, indent=2)

print(f"✓ Created profile with {len(syscalls)} allowed syscalls")
```

[SAVE FILE - Ctrl+S]

[SCREEN: Terminal]

[TYPE:]

```
python3 build_profile.py
```

[OUTPUT:]

```
✓ Created profile with 47 allowed syscalls
```

NARRATION:
“Excellent! We’ve created a Seccomp profile with just 47 allowed syscalls—an 81% reduction from Docker’s default.”

[SCREEN: Code Editor - open seccomp-nginx.json]

[SCROLL through the file slowly]

NARRATION:
“Here’s our complete profile. Look at the syscalls: accept4, bind, brk, clone, close, epoll_wait, read, write... these are exactly what nginx needs to function. Nothing more, nothing less.”

SEGMENT 5: Step 3 - Testing the Profile (10:00 - 12:30)

[SCREEN: Terminal]

NARRATION:
“Time to test! Let’s run nginx with our custom profile and see if it works.”

[TYPE:]

```
docker run --rm --security-opt seccomp=seccomp-nginx.json -p 8080:80 nginx:alpine
```

[COMMAND RUNS - nginx starts successfully]

NARRATION:
“It started! Now let’s test if it actually serves web pages.”

[SCREEN: Terminal split - nginx running on left, curl on right]

[RIGHT PANE - TYPE:]

```
curl http://localhost:8080
```

[HTML OUTPUT shows - nginx welcome page]

NARRATION:
“Perfect! We get the nginx welcome page. Our profile works. Let me test a few more operations to make sure.”

[TYPE:]

```
curl -I http://localhost:8080
```

[HEADERS display successfully]

[TYPE:]

```
for i in {1..10}; do curl -s http://localhost:8080 > /dev/null && echo "Request $i:
OK"; done
```

[OUTPUT shows 10 successful requests]

NARRATION:
“All requests successful. Our minimal profile allows nginx to operate normally while blocking 80% of syscalls. This dramatically reduces what an attacker can do if they compromise nginx.”

SEGMENT 6: Handling Edge Cases (12:30 - 14:30)

[SCREEN: Terminal]

NARRATION:
“But what if our profile is too restrictive? Let me show you what happens when we accidentally block something nginx needs.”

[SCREEN: Code Editor - seccomp-nginx.json]

NARRATION:
“I’m going to manually remove the `epoll_wait` syscall from our profile. nginx uses this for event handling, so this should break things.”

[EDIT the file - remove epoll_wait from the names array]

[SAVE]

[SCREEN: Terminal]

[TYPE:]

```
docker run --rm --security-opt seccomp=seccomp-nginx.json nginx:alpine
```

[CONTAINER STARTS but then FAILS or HANGS]

NARRATION:
“See? nginx can’t function without epoll_wait. But the error message isn’t very clear. This is where debugging gets tricky. In the next video, we’ll cover advanced troubleshooting techniques.”

[STOP the container - Ctrl+C]

NARRATION:
“For now, let me restore epoll_wait and show you a best practice: testing incrementally.”

[SCREEN: Code Editor - restore epoll_wait]

NARRATION:
“When you build custom profiles, test each application feature:
- Basic startup
- Handling requests
- Logging
- Signal handling
- Graceful shutdown

If anything breaks, check your syscalls. Often you’ll discover edge cases that only appear under specific conditions—like nginx reloading configuration or handling SSL connections.”

SEGMENT 7: Lab Exercise - Python Flask App (14:30 - 15:30)

[SCREEN: PPT Slide 4 - Lab overview]

NARRATION:
“Now it’s your turn. In Lab 3.2, you’ll create custom Seccomp profiles for:

1. A Python Flask web application
I've provided a sample Flask app. You'll:
- Run it with strace
- Extract syscalls
- Build a custom profile
- Test and validate

2. A multi-process application
You'll handle an app that spawns child processes and needs different syscalls at different lifecycle stages.

You'll use everything we covered: strace, profile building, testing, and debugging.

The lab includes validation scripts that will check:
- ✓ Profile syntax is valid
- ✓ Application runs successfully
- ✓ No unnecessary syscalls are allowed
- ✓ Key functionality works (HTTP requests, database access, etc.)

SEGMENT 8: Wrap-Up (15:30 - 16:00)

[SCREEN: PPT Slide 5 - Key takeaways]

NARRATION:
"Let's recap what we've learned:

✅ **strace** is your primary tool for discovering syscalls
✅ Custom profiles can reduce attack surface by **80%+**
✅ Profile structure: defaultAction, architectures, syscalls array
✅ Always test thoroughly before production
✅ Document which features you've tested

In the next video, we'll dive into advanced troubleshooting: reading audit logs, debugging Seccomp denials, and strategies for maintaining profiles as applications evolve.

See you there!"

[FADE OUT]

POST-PRODUCTION NOTES

B-Roll Suggestions:

- Screen recording of strace output scrolling (sped up 3x)
- Animated JSON structure being built
- Side-by-side comparison: default profile vs custom (syscall count visualization)
- Graph showing attack surface reduction

Audio Notes:

- Background music during intro/outro only

- Clear typing sounds during code sections
- Success “ding” when tests pass
- Add echo effect when saying “80% reduction” for emphasis

Text Overlays:

- Syscall count comparisons (250+ → 47)
- Command explanations during complex bash commands
- Highlight key JSON fields when first introduced
- Step numbers (1/4, 2/4, etc.) during workflow section

Timing Checkpoints:

- 0:00-3:00: Intro + Workflow (3 min)
- 3:00-6:30: strace Discovery (3.5 min)
- 6:30-10:00: Profile Building (3.5 min)
- 10:00-12:30: Testing (2.5 min)
- 12:30-15:30: Edge Cases + Lab (3 min)
- 15:30-16:00: Wrap-up (0.5 min)
- **Total: 16:00** (within 14-16 min target)

Files Needed:

- `slides_module3_video2.pptx` (5 slides)
- `nginx_syscalls.txt` (example output)
- `seccomp-nginx.json` (completed profile)
- `build_profile.py` (automation script)
- `flask_sample_app.py` (for lab)
- `lab_3.2_instructions.md`
- `lab_3.2_validation.sh`

INSTRUCTOR NOTES

Common Issues:
- strace may not be available in some minimal containers - install with package manager
- `-f` flag in strace is critical for multi-process apps like nginx
- Some distributions limit strace usage - may need `--cap-add SYS_PTRACE`
- pbcopy command is macOS-specific; use xclip on Linux or manual copy

Variation Options:
- Can use `seccomp-tools` for automated profile generation
- Can demonstrate with alternative apps (Redis, PostgreSQL, Node.js)
- Can show Docker Compose integration
- Advanced: show how to use bpftrace instead of strace

Accessibility:
- Slow down when typing complex commands
- Read JSON structure aloud
- Pause 3 seconds after each command output
- Use zoom-in on terminal when showing specific syscall names

Pro Tips for Delivery:

- Have all files pre-created as backup in case live demo fails
- Test commands before recording
- Keep a “cheat sheet” of syscalls to discuss if time permits
- Mention real-world case: how companies like Google use custom Seccomp profiles